TJA1145A Driver Software Design

> **Version**: 1.2
> **Date**: 2026-03-12
> **Scope**: S32G399A M7_0 current TJA1145A bare-metal driver design and integration behavior

1. Document Overview

This document describes the actual design approach used by the current TJA1145A driver in this project. The focus is not to restate the entire chip manual, but to explain how the current code actually works, why it is implemented this way, and how it cooperates with FlexCAN.

The document follows a top-down structure. It first explains the system position and overall flow, then drills down into SPI access, register strategy, and recovery logic. This helps readers build a complete picture before moving into implementation details.

1.1 Reading Guide

Chapter	Content	Target Audience
Chapter 2	File List	All readers, for quickly locating related code
Chapter 3	System Global View	All readers, to first understand the driver's position in the system
Chapter 4	Module Architecture	Developers, to understand module boundaries and responsibilities
Chapter 5	Core Process Design	Developers, to understand the main flows for initialization, monitoring, and recovery
Chapter 6	Module Detailed Design	Developers, to review SPI, register, and interface details
Chapter 7	Integration and Constraints	Integrators, focusing on coordination with FlexCAN and the related risks
Chapter 8	Appendix	Debug and maintenance personnel, for quickly checking key registers

1.2 Document Scope

This document only covers the content that has already been implemented in the current project, or is directly depended on by the current implementation:

- Bare-metal access to TJA1145A through DSPI5.
- The TJA1145A mode bring-up flow during the startup phase.
- Runtime status monitoring and recovery logic executed about every 100 ms.
- The current direct interaction between this driver and FlexCAN0.

This document does not expand on the following topics:

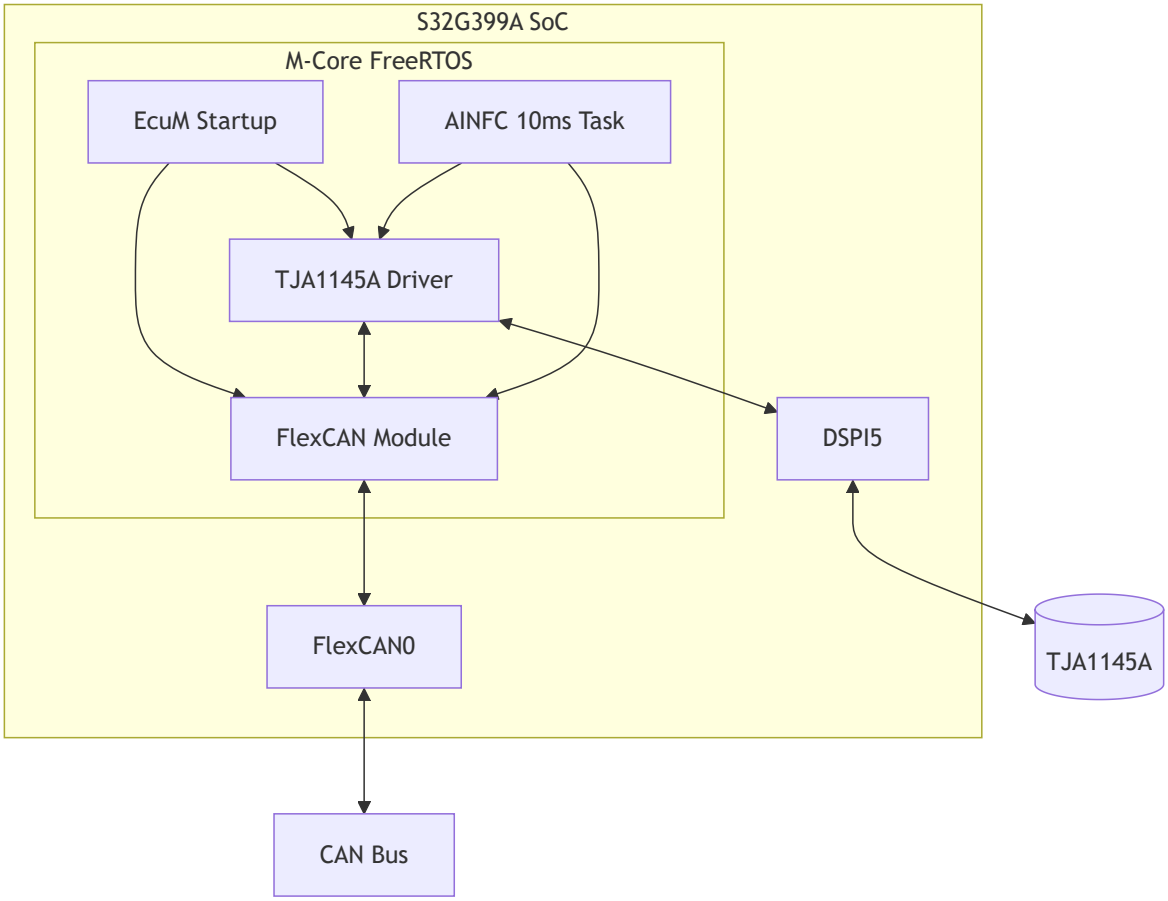
- The complete feature set from the full TJA1145A manual.
- A full configuration scheme for selective wake-up and partial networking.
- A generalized driver framework design for multi-instance or multi-controller support.
- A standard diagnostic architecture for production-level product deployment.

2. File List

File	Type	Description
`TJA145A_Spi_Ip/TJA1145A_Spi_Baremetal.h`	Header	Interface declarations for DSPI5 and TJA1145A, plus key register and debug structure definitions
`TJA145A_Spi_Ip/TJA1145A_Spi_Baremetal.c`	Source	Main implementation of SPI initialization, register read/write, TJA initialization, recovery, and periodic checking
`src/Ecum_Init_Main/EcuM_main_init.c`	Source	Call entry point for SPI, TJA, and FlexCAN during system startup
`FlexCAN_Ip/FlexCAN_Ip_main.c`	Source	10 ms periodic task and the trigger point for TJA periodic checks
`FlexCAN_Ip/FlexCAN_Ip_main.h`	Header	Current CAN module interfaces, MB allocation, and application definitions
`S32G3_reference_manual/TJA1145A.md`	Reference	Manual-based reference for TJA1145A modes, registers, and wake-up constraints

3. System Global View

3.1 System Positioning



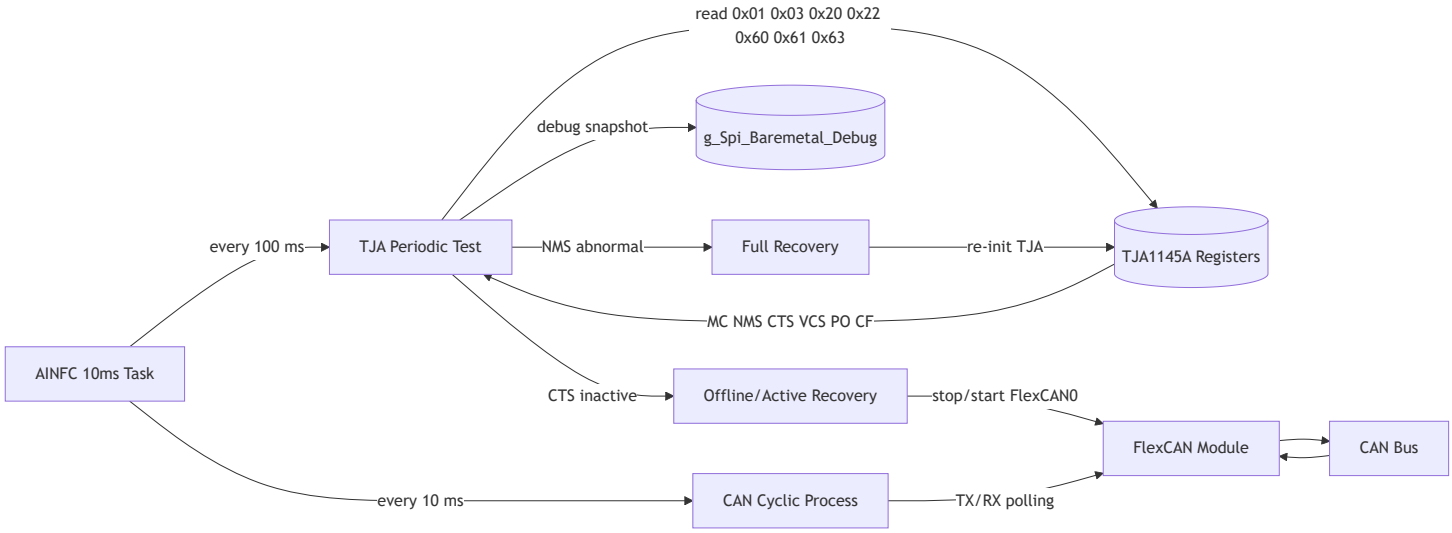
This diagram first answers the most fundamental question: where the TJA1145A driver sits in the system.

As shown, the current design does not treat TJA1145A as a completely isolated low-level device. Instead, it is placed between the startup flow, the periodic task, and the FlexCAN controller so they work together. In other words, it serves both as an SPI-access component and as a CAN transceiver state-management component.

Several connections in the diagram deserve special attention:

1. `EcuM Startup -> TJA1145A Driver`
   - This means the TJA1145A is explicitly brought up by the system initialization flow during startup.
   - This step is not completed automatically in the background; it is explicitly called by the startup code.
2. `TJA1145A Driver <-> DSPI5 <-> TJA1145A`
   - This means all mode transitions and status collection depend on DSPI5 register access.
   - There is currently no intermediate abstraction layer; the driver directly accesses DSPI5 registers.
3. `TJA1145A Driver <-> FlexCAN Module`
   - This means the current recovery logic is directly coupled with the CAN controller.
   - When the transceiver becomes inactive, the driver directly stops and starts FlexCAN0 instead of only returning an error to an upper layer.
4. `AINFC 10ms Task -> TJA1145A Driver`
   - This means runtime monitoring is not interrupt-driven, but executed from a periodic task.
   - In the current project, the transceiver status is checked approximately every 100 ms.

3.2 Runtime Data Flow



This diagram describes the runtime data flow that actually happens, not just the static structure. It emphasizes two facts:

- First, application message transmission/reception and TJA status checking share the same periodic task context.
- Second, the current recovery logic can affect the FlexCAN module in the reverse direction instead of staying fully contained inside the TJA driver.

When reading this diagram, it is helpful to follow the sequence below:

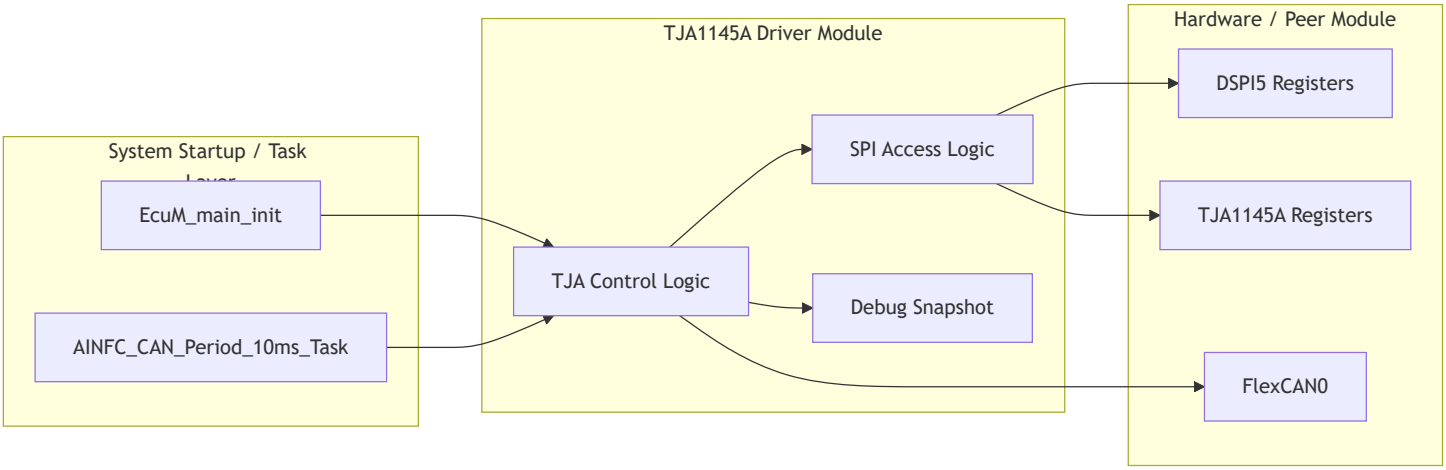
1. `AINFC 10ms Task` on the left is the unified entry point.
   - One branch goes into `CAN Cyclic Process`, which handles normal TX/RX processing.
   - The other branch periodically goes into `TJA Periodic Test`, which performs status checking.
2. `TJA Periodic Test <-> Registers` in the middle means the monitor function actively reads key registers.
   - `Mode Control (0x01)` is used to confirm the current written mode value.
   - `Main Status (0x03)` is used to determine whether the device is still in Normal mode.
   - `CAN Control (0x20)` is used to confirm the current CMC value.
   - `Transceiver Status (0x22)` is used to determine whether the transceiver is actually active.

- `System/Transceiver Event (0x60/0x61/0x63)` is used for diagnostic observation.

3. `debug snapshot` means every monitoring cycle updates the global debug image.
   - During debugging, engineers can directly inspect this snapshot instead of stopping execution and reading the live registers every time.
4. `Full Recovery` and `Offline/Active Recovery` are the two recovery actions.
   - The former handles mode abnormalities. It is a heavier action and re-enters the main initialization path.
   - The latter handles `CTS=0`. It is lighter and only redoes CAN Control while coordinating with FlexCAN0.

4. Module Architecture

4.1 Layered Structure



This diagram shows that the current driver internally contains three parts: control logic, SPI access logic, and debug-visible data.

It is not a strict layered framework in the formal sense. Instead, it is an integrated module built around a single device. The benefit is that the current code is compact and straightforward for debugging. The cost is that responsibility boundaries are tight, so if multi-instance support or platform refactoring is needed later, the restructuring cost will be higher.

The three internal blocks in the diagram mean the following:

1. `TJA Control Logic`
   - Responsible for initialization sequence, mode switching, periodic checking, and recovery decisions.
   - This is the true behavioral center of the driver.
2. `SPI Access Logic`
   - Responsible for DSPI5 initialization, single-byte transfer, and register read/write wrapping.
   - It provides the most basic communication capability for the control logic.
3. `Debug Snapshot`
   - Responsible for recording key register images, status summaries, and error codes.
   - The current driver keeps this part as globally visible data for easier debugging.

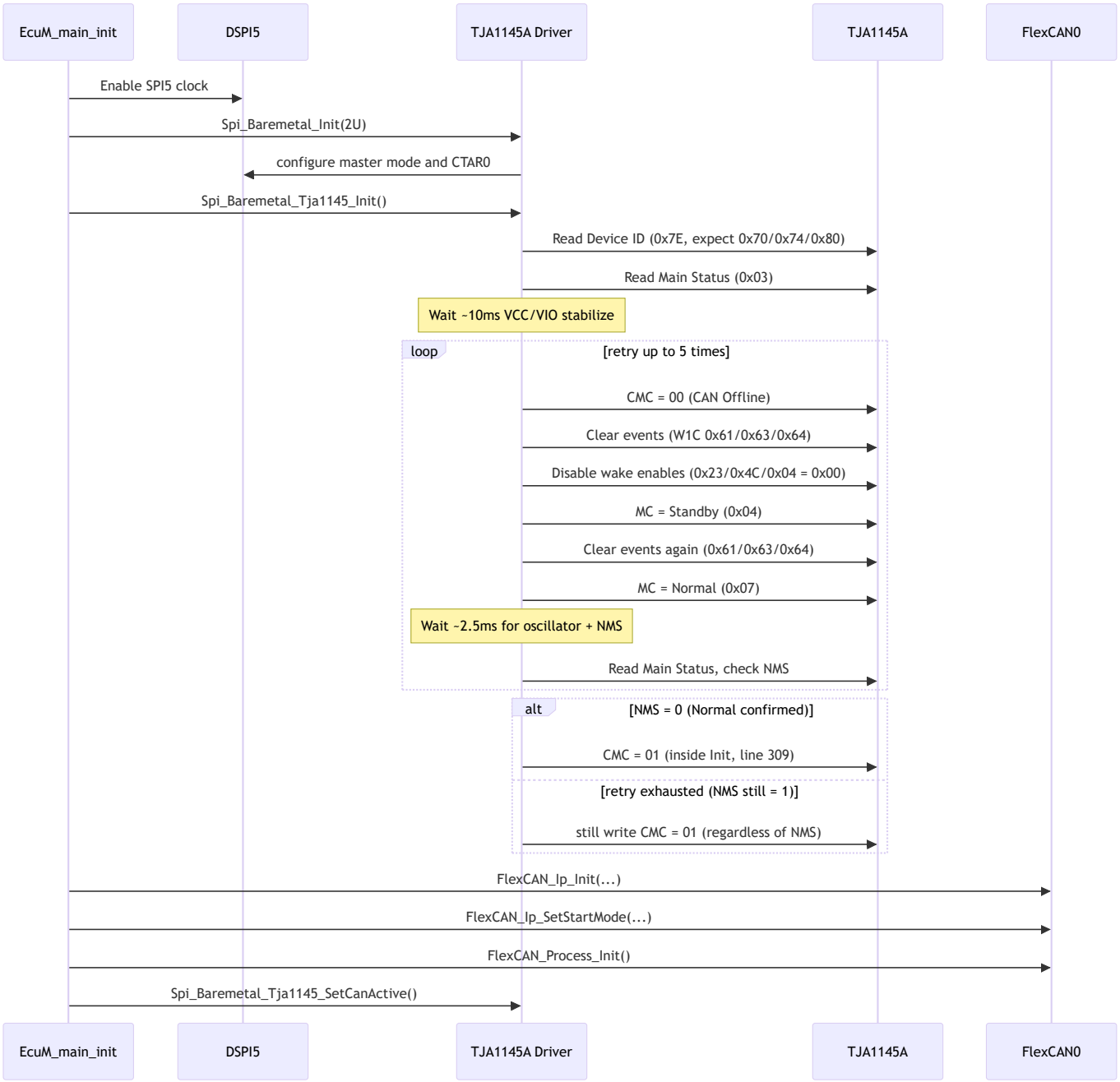
4.2 Module Responsibility Overview

Area	Responsibility	Current Implementation Characteristic
SPI Access	Initialize DSPI5, execute 8-bit SPI transfer, and assemble register read/write sequences	Polling-based, no DMA, no interrupt, direct register access

Area	Responsibility	Current Implementation Characteristic
TJA Control	Initialize TJA1145A, switch Normal/Standby, set CAN Active	Targeted to the current single device, with the flow written directly in one source file
Runtime Monitor	Periodically read status registers and detect `NMS` and `CTS` abnormalities	Triggered every 100 ms by the 10 ms task
Recovery	Full recovery on mode abnormality, light recovery when the transceiver becomes inactive	Directly coupled to FlexCAN0 stop/start
Debug	Store register snapshots, error codes, and supply-related status summaries	Convenient for TRACE32 observation, but with weaker encapsulation boundaries

5. Core Process Design

5.1 Initialization Sequence



This is the most important diagram in the document because it fully connects what actually happens during startup.

The current initialization can be understood in two parts:

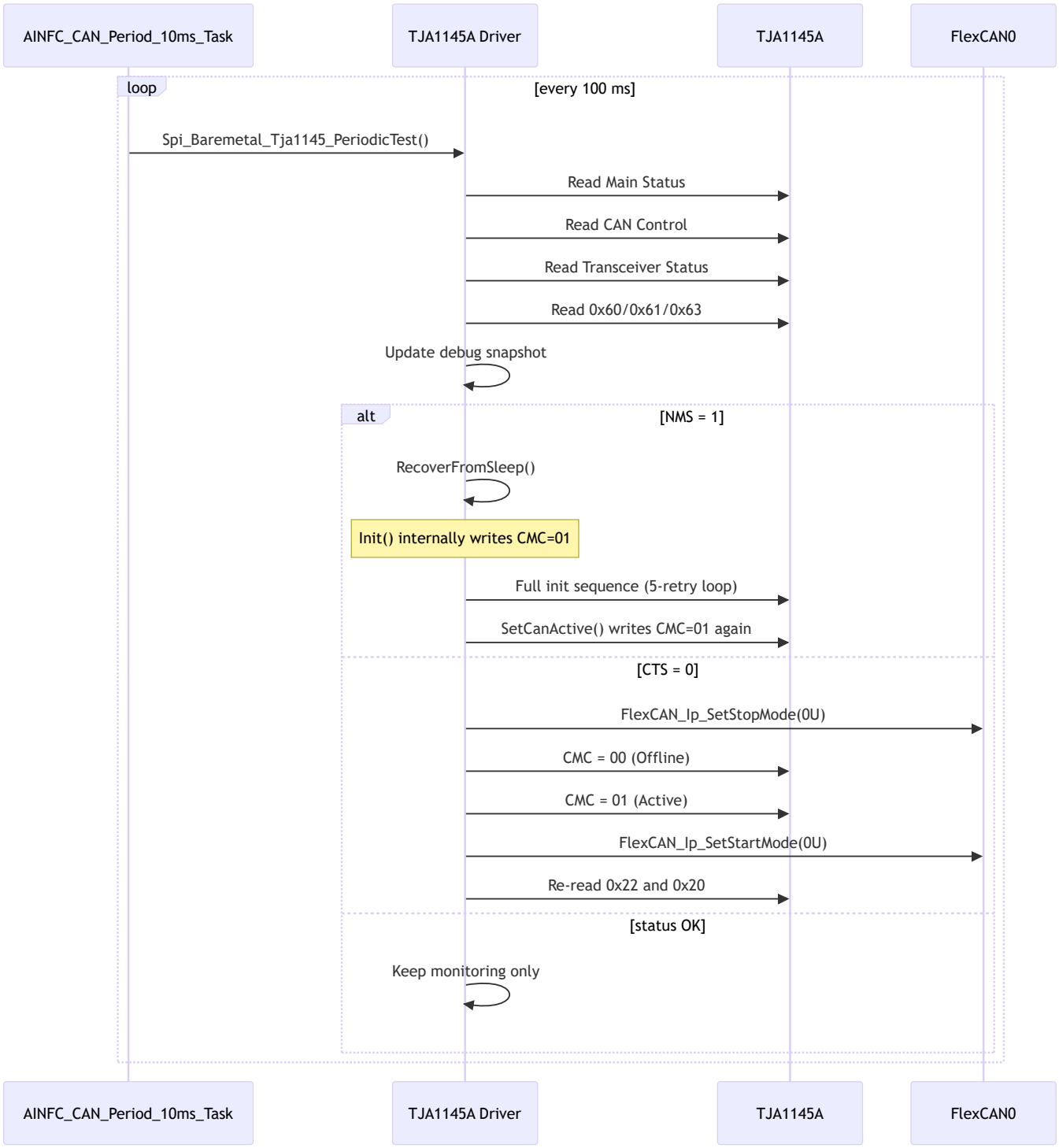
1. Prepare the SPI channel first.
   - If DSPI5 itself is not brought into the correct operating state, then all subsequent register access is invalid.
2. Then bring TJA1145A from an uncertain power-on state into a working state acceptable to the current software, with up to 5 retries.
   - First force `CAN Offline` (`CMC=00`) to hold the bus-side behavior in a controlled state.
   - Then call `ClearEventsAndDisableWakeup()` . This function first clears the three event registers 0x61/0x63/0x64 using W1C, and then writes `0x00` to 0x23 (`Transceiver Event Enable`), 0x4C (`Wake Pin Enable`), and 0x04 (`System Event Enable`) to disable all wake-up sources.
   - Then follow the path `Standby -> Clear events again -> Normal` .

One implementation characteristic must be stated explicitly here:

- The current code uses `NMS=0` as the core condition for a successful initialization.
- However, even if that condition is not met in the end, the function still attempts to write `CAN Control` to `0x01` before it exits.

This means "initialization returns failure" and "the driver still tries to bring the transceiver up" can both happen at the same time in the current implementation. During debugging, the link state cannot be judged only from the return value.

5.2 Runtime Monitoring and Recovery



This diagram answers three runtime questions: when checking happens, what is checked, and how the code reacts after an abnormal condition is detected.

First, look at the trigger model:

- Periodic checking is not done inside an interrupt, but in the CAN periodic task.
- The current task runs every 10 ms, but TJA checking is not executed every 10 ms. Instead, it runs at a lower frequency of about 100 ms.

Then look at the check contents:

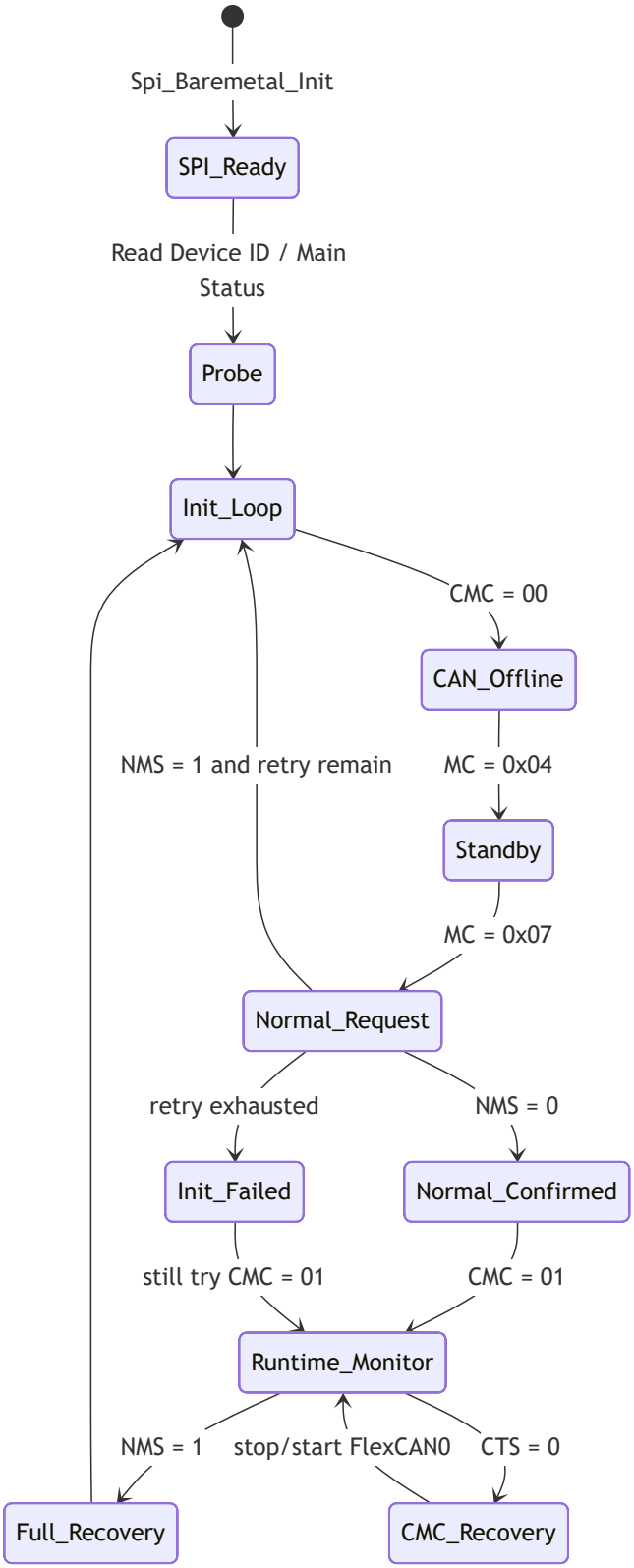
- `Mode Control (0x01)` reads back the current mode value and stores it in the debug image for confirmation during debugging.
- `Main Status (0x03)` mainly checks `NMS` to determine whether the device has left Normal mode.
- `CAN Control (0x20)` reads back the current CMC value and records it in the debug image.
- `Transceiver Status (0x22)` mainly checks `CTS` to determine whether the transceiver is truly active.
- `0x60/0x61/0x63` is mainly used for diagnostic observation (`FSMS` , `VCS` , `PO` , `CF` count, and supply status), and does not directly decide the business path.

Finally, look at the recovery actions:

1. `NMS = 1`
   - This is treated as a more severe fault.
   - The current strategy is to perform a full recovery, which is equivalent to re-initializing the TJA.
2. `CTS = 0`
   - This is treated as a lighter fault.
   - The current strategy is not a full restart. Instead, it stops FlexCAN0 first, switches `CMC` from Offline back to Active, and then restarts FlexCAN0.
3. `status OK`
   - Only the monitoring snapshot is updated, with no extra action.

This also shows a design reality: the current driver is no longer just a TJA register read/write wrapper. It already takes on part of the link-recovery coordination responsibility.

5.3 Software State Machine



This diagram is not the full hardware state machine from the manual. It is the state machine that is actually implemented by the current software. That distinction must be kept clear, otherwise the chip capability and the software behavior are easily mixed together.

To understand this diagram, it is useful to divide it into three stages:

1. Startup link-establishment stage
    - `SPI_Ready -> Probe -> Init_Loop`

- The goal is to prove that SPI is usable, the device is accessible, and the mode bring-up flow can begin.

2. Initialization confirmation stage
   - `CAN_Offline -> Standby -> Normal_Request -> Normal_Confirmed`
   - The goal is to bring the TJA into a state the current software considers usable.
   - The criterion is simple: `NMS=0` .
3. Runtime monitoring stage
   - `Runtime_Monitor`
   - After entering this stage, the driver no longer actively switches among many complex modes. It only performs two checks: `NMS` abnormality and `CTS` abnormality.

There is another important detail in the diagram:

- `Init_Failed -> Runtime_Monitor : still try CMC = 01`

This line explicitly reflects the real behavior of the current code: even if Normal confirmation fails, it still tries to make the transceiver active. This behavior can easily cause misjudgment during debugging, so it should be shown directly in the state machine instead of being mentioned only in the text.

6. Module Detailed Design

6.1 SPI Access Design

The current SPI access layer only serves one target device, so the design is direct and does not introduce additional bus abstraction.

The core responsibilities of `Spi_Baremetal_Init` are as follows:

1. Put DSPI5 into a clean and controllable Master mode.
2. Configure CTAR0 for 8-bit frame, `CPOL=0` , and `CPHA=1` .
3. Disable DMA and interrupts, and use polling for transmission and reception.
4. Record the initialization result and status-bit snapshots in the debug structure.

The main characteristics of `Spi_Baremetal_TransferEx` are as follows:

- It transfers only one byte per call.
- It uses `cont_cs` to control whether chip select remains asserted.
- It waits for TX/RX completion by polling.
- On timeout, it returns `0xFF` and updates the error code.

This means the assumptions behind the current SPI-access design are very explicit:

- DSPI5 is exclusively owned by the current module during runtime.
- The current real-time budget allows busy-wait polling.
- At the current stage, debugging simplicity and deterministic behavior are prioritized over optimal transfer efficiency.

6.2 Register Strategy

The current implementation actually depends on only a small set of key registers. The table below is more useful in practice than a complete register map.

Register	Addr	Current Usage	Why It Matters
Mode Control	0x01	Write Standby/Normal/Sleep mode values; also read back in `PeriodicTest`	Determines the main chip mode transition, and is read back at runtime for debug confirmation
Main Status	0x03	Read `NMS`, `FSMS`, `OTWS`	Determines whether the chip is in Normal, and whether there are signs of forced sleep due to undervoltage
System Event Enable	0x04	Written as 0x00 during initialization to disable it	Prevents system events from triggering wake-up
CAN Control	0x20	Read and write `CMC`	Determines whether the transceiver is Offline or Active
Transceiver Status	0x22	Read `CTS`, `VCS`	Determines whether the transceiver is truly active and the VCC supply state
Transceiver Event Enable	0x23	Written as 0x00 during initialization to disable it	Prevents transceiver events from triggering wake-up
Wake Pin Enable	0x4C	Written as 0x00 during initialization to disable it	Prevents the Wake Pin from triggering wake-up
Global Event Status	0x60	Read at runtime for diagnostics	Used to observe global event flags for diagnostics
System Event Status	0x61	Cleared with W1C during initialization, read at runtime for diagnostics	Used to observe events such as `PO`
Transceiver Event Status	0x63	Cleared with W1C during initialization, read at runtime for diagnostics	Used to observe transceiver events such as `CF`
Wake Pin Event	0x64	Cleared with W1C during initialization	Prevents historical wake events from affecting mode transitions
Device ID	0x7E	Read during startup to identify the chip (expected 0x70/0x74/0x80)	Verifies that SPI communication is fundamentally working

The bit semantics that the current code actually depends on can be reduced to the following:

- `Main Status[5] = NMS`
  - The current software uses this as the primary criterion for whether Normal mode has been entered.
- `Main Status[7] = FSMS`
  - Used to observe whether the chip was forced into Sleep because of undervoltage.
- `CAN Control[1:0] = CMC`
  - The current code actually uses only `00 = Offline` and `01 = Active`.
- `Transceiver Status[7] = CTS`
  - Indicates whether the transceiver is actually active.
  - This is closer to the real hardware state than the fact that the software just wrote `CMC=01`.
- `Transceiver Status[1] = VCS`
  - Used to observe whether VCC is in the undervoltage range.

6.3 Wake, Sleep and Recovery Policy

The current code treats low-power behavior with the attitude of "avoid complexity first, then make sure bring-up works."

This is reflected as follows:

1. During initialization, wake sources are disabled and related events are cleared.
   - The purpose is not to implement a low-power strategy, but to avoid historical events interfering with entry into Normal mode.
2. The main flow does not actively enter Sleep.
   - The current code focuses on starting communication and keeping communication alive, not on implementing product-grade sleep/wake features.
3. Runtime logic performs only two types of recovery.
   - Full recovery when `NMS` is abnormal.
   - Light recovery when `CTS` is abnormal.

The advantage of this strategy is that the logic is simple and debugging is direct. The disadvantage is that the recovery action is relatively coarse. If future requirements need to distinguish among command-triggered sleep, undervoltage forced sleep, wake-event interference, and other scenarios, additional branch policies will be required.

6.4 External Interface Summary

API	Purpose	Key Input/Output	Notes
`Spi_Baremetal_Init(uint8 baudrate_div)`	Initialize DSPI5	Input divider value	The SPI5 clock must already be enabled
`Spi_Baremetal_TransferEx(uint8 tx_byte, uint8 cont_cs)`	Send one SPI byte	Returns the received byte	Returns `0xFF` on timeout
`Spi_Baremetal_Tja1145_ReadReg(uint8 addr)`	Read one register	Returns the register value	Internally performs a two-byte SPI access
`Spi_Baremetal_Tja1145_WriteReg(uint8 addr, uint8 data)`	Write one register	No return value	Write success is mainly judged through later readback or observed behavior
`Spi_Baremetal_Tja1145_Init(void)`	Initialize TJA1145A with up to 5 retries	Returns `0` = success, `1` = NMS not confirmed	Even on failure it still writes `CMC=01` (line 309)
`Spi_Baremetal_Tja1145_RecoverFromSleep(void)`	Bring TJA1145A back up again	Returns `0` = success, `1` = failure	Internally calls `Init()` + `SetCanActive()`, so `CMC=01` is written twice
`Spi_Baremetal_Tja1145_SetCanActive(void)`	Put the transceiver into Active state	No return value	After writing `CMC=0x01`, it reads back 0x22 and 0x20 to update the debug image
`Spi_Baremetal_Tja1145_PeriodicTest(void)`	Periodic monitoring and recovery	No return value	Reads 7 registers: 0x01/0x03/0x20/0x22/0x60/0x61/0x63

7. Integration and Constraints

7.1 Current Relation with FlexCAN

The relationship between the current driver and FlexCAN can be summarized in three sentences:

1. FlexCAN is responsible for the actual message transmission and reception.
2. TJA1145A is responsible for transceiver mode and hardware link availability.
3. In the current recovery code, the TJA driver directly coordinates FlexCAN0 stop/start.

This design works in the current project because the path is short and issue localization is fast. But it also brings a very practical constraint:

- The TJA driver already knows the FlexCAN instance number and the recovery sequence.
- If future work needs to support multiple controllers or a clearer software layering model, this direct coupling point will need to be split out.

So the current recommendation is:

- Keep the existing implementation for debugging and the current project delivery.
- If the CAN architecture enters a second-stage convergence later, extract the "recovery coordination responsibility" from the TJA driver into an independent coordination layer.

7.2 Assumptions and Risks

The current design is built on the following assumptions:

- DSPI5 will not be preempted by other modules while the driver is running.
- The scheduling of the 10 ms periodic task is sufficient to absorb status checking and recovery actions.
- The current system accepts the CPU usage introduced by polling-based SPI access.
- Only one TJA1145A instance and one FlexCAN0 instance need to be maintained at present.

The main risks that need close attention are as follows:

1. `NMS` is the only primary criterion.
   - The current logic mainly relies on `NMS` to determine whether Normal mode was entered successfully.
   - If stricter confirmation is required later, more status bits will need to be checked together.
2. The code still attempts `CMC=01` after initialization failure.
   - This can make "initialization not fully successful" and "the link appears partially recovered" exist at the same time.
   - During debugging, this must be judged together with `CTS` and real bus measurements.
3. Recovery actions share the same path as the application task.
   - If recovery actions become heavier, they may affect the execution rhythm of the periodic application logic.
4. The driver is directly coupled with FlexCAN.
   - This is convenient for troubleshooting today, but not ideal for future modular evolution.

8. Appendix

8.1 Key Mode Values

Symbol	Value	Meaning
`TJA1145_MODE_SLEEP`	`0x01`	Sleep mode
`TJA1145_MODE_STANDBY`	`0x04`	Standby mode
`TJA1145_MODE_NORMAL`	`0x07`	Normal mode

8.2 Key Diagnostic Bits

Register.Bit	Meaning	Current Use
`Main Status[7] FSMS`	forced sleep indication	Tracks signs of forced sleep caused by undervoltage
`Main Status[5] NMS`	not in Normal mode	Primary criterion during initialization and runtime
`Transceiver Status[7] CTS`	transceiver active state	Determines whether light recovery is needed
`Transceiver Status[1] VCS`	VCC undervoltage state	Records signs of supply abnormality
`System Event Status[4] PO`	power-on event	Used for diagnostic observation
`Transceiver Event Status[1] CF`	CAN failure event	Used for diagnostic observation

8.3 Current CAN Mapping

Item	Current Setting
FlexCAN instance	`0U`
TX MB	MB0, MB1
RX MB	MB2, MB3
Periodic task	`AINFC_CAN_Period_10ms_Task`
TJA periodic test rate	about every 100 ms